Q 1. Analyze the architectural trade-offs between using an **Arduino Uno** versus a **Raspberry Pi 4** for a remote, battery-powered weather station that must perform complex data encryption before uploading it to the cloud. Which board is more suitable for this specific constraint?

Ans:

Connectivity Type	Best Boards	Range	Power Use	Ideal Use Case
Wi-Fi	ESP32, Raspberry Pi 4, Photon 2	~50-100m	High	High-speed data, plugged-in smart home devices.
Bluetooth (BLE)	Arduino Nano 33 IoT, ESP32-S3	~10-30m	Low	Wearables, local smartphone control, "Find My" tags.
Cellular (LTE/NB-IoT)	Particle Boron, Arduino MKR NB 1500	Global	Medium	Remote assets (GPS trackers, rural weather stations).
Ethernet (Wired)	Raspberry Pi 4, BeagleBone Black	Cable limit	High	Fixed gateways, industrial servers, high-security links.

For a **remote, battery-powered** station that must perform **complex encryption**, the **Raspberry Pi 4** is the more suitable board for the *functional* requirement (encryption and cloud upload), but the **Arduino Uno** is better for the *environmental* constraint (battery life).

Q 2. Elaborate in short Health & Lifestyle domain specific IoT.

Ans: **Core Role in Health & Lifestyle**

- **Continuous Vitals Monitoring:** Devices track metrics like heart rate, blood pressure, oxygen and glucose levels in real-time.
  - **Preventative Care & Alerts:** IoT platforms use rule engines to detect anomalies (e.g., a sudden drop in glucose or irregular heart rhythm) and instantly alert doctors or family members.
  - **Lifestyle Management:** Fitness trackers and smartwatches monitor physical activity, sleep patterns, and even mood to help users make informed daily health choices.
  - **Medication Adherence:** Smart inhalers and connected pill dispensers track usage and send reminders, ensuring patients follow prescribed treatment plans.

Key Domain-Specific Applications

- **Chronic Disease Management:** specialized tools like **Continuous Glucose Monitors (CGM)** for diabetes or **connected inhalers** for asthma allow for long-term tracking without frequent hospital visits.
- **Elderly Care & Fall Detection:** Wearables equipped with accelerometers can detect falls and notify emergency services automatically.
- **Advanced Diagnostics:** Emerging tech includes **ingestible sensors** (pills that track internal health) and **smart contact lenses** that monitor glucose through tears.

Q 3. Analyze why the **overhead** associated with a TCP "Three-Way Handshake" might make it unsuitable for an IoT sensor powered by a small coin-cell battery that only transmits 10 bytes of data once per day. Suggest an alternative and justify your choice.

- Ans: TCP is unsuitable because the handshake and acknowledgement packets consume significantly more power than the actual 10-byte payload, leading to **excessive energy drain**. **UDP** is the better alternative because it is **connectionless**, requiring no handshake, thus minimizing radio "on-time" and preserving battery life.

Q 4. Elaborate the working model of smart city.

Ans: **Core Components of the Model**

- **Smart Infrastructure:** Upgrading traditional assets (grids, pipes, and roads) with IoT capabilities to monitor their own health and usage [8].
- **Edge & Cloud Computing:** Critical data (like autonomous vehicle signals) is processed "at the edge" for instant response, while long-term data is sent to the cloud for urban planning [6, 7].
- **Open Data Portals:** A transparent model where data is shared with citizens and developers to create third-party apps, such as real-time bus tracking or energy-saving dashboards [10].
- **Citizen Engagement:** The "human" element where residents use mobile apps to report potholes, pay bills, or participate in local governance [10, 11].

Typical "Smart" Use Cases

Sector	Working Example
Mobility	Adaptive traffic lights that change timing based on real-time vehicle flow [9].
Energy	Smart grids that automatically shift power loads to renewable sources during peak hours [12].
Waste	Trash bins with ultrasonic sensors that alert collectors only when they are 90% full

Q 5. Describe 10 applications of IoT in the market place.

1. Ans: Home Automation Smart Lighting, Smart Appliances, Intrusion Detection, Smoke/Gas Detectors.
2. Cities Smart Parking, Smart Lighting, Smart Roads, Structural Health
3. Environment Weather Monitoring, Air Pollution Monitoring, Noise Pollution Monitoring, Forest Fire Detection, Monitoring, Surveillance. River Floods Detection.
4. Environment Weather Monitoring, Air Pollution Monitoring, Noise Pollution Monitoring, Forest Fire Detection, River Floods Detection.
5. Energy Smart Grids, Renewable Energy Systems, Prognostics.
6. Retail Inventory Management, Smart Payments, Smart Vending Machines.
7. Logistics Route Generation & Scheduling, Fleet Tracking, Shipment Monitoring.
8. Agriculture Smart Irrigation, Green House Control.
9. Industry Machine Diagnostics & Prognosis, Indoor Air Quality Monitoring.

Health & Lifestyle Health & Fitness Monitoring, Wearable Electronics.

Q 6. Compare different IoT boards in terms of connectivity.

Ans:

Connectivity Type	Best Boards	Range	Power Use	Ideal Use Case
Wi-Fi	ESP32, Raspberry Pi 4, Photon 2	~50-100m	High	High-speed data, plugged-in smart home devices.
Bluetooth (BLE)	Arduino Nano 33 IoT, ESP32-S3	~10-30m	Low	Wearables, local smartphone control, "Find My" tags.
Cellular (LTE/NB-IoT)	Particle Boron, Arduino MKR NB 1500	Global	Medium	Remote assets (GPS trackers, rural weather stations).
Ethernet (Wired)	Raspberry Pi 4, BeagleBone Black	Cable limit	High	Fixed gateways, industrial servers, high-security links.

Q 7. Analyze the role of IoT Software Platforms by addressing the following:

1. Device Management and Connectivity
2. Data Normalization and Integration

3. Data Processing and Rule Engines
4. Application Enablement (Low-Code/No-Code)
5. Security and Encryption

Ans. IoT software platforms act as the "connective tissue" or "operating system" of an IoT ecosystem. They sit between the physical hardware (sensors/actuators) and the application layer, simplifying the complexity of managing thousands of devices.

1. Device Management and Connectivity

The platform serves as a central hub to **provision**, **authenticate**, and **configure** devices. It ensures that hardware can communicate securely with the cloud or other devices, regardless of whether they use Wi-Fi, Bluetooth, Cellular, or LoRaWAN. It also handles critical "over-the-air" (OTA) firmware updates to keep devices secure and functional.

2. Data Normalization and Integration

IoT devices often speak different "languages" (protocols like MQTT, CoAP, or HTTP). The platform translates this fragmented data into a **unified format**. This allows the data to be easily integrated with existing enterprise systems, such as an ERP or CRM, ensuring that sensor data actually informs business decisions.

3. Data Processing and Rule Engines

Instead of just storing raw data, platforms use **rule engines** to trigger actions. For example, if a temperature sensor exceeds 50°C, the platform can automatically send an alert to a technician or shut down a machine. This enables real-time automation without human intervention.

4. Application Enablement (Low-Code/No-Code)

Platforms provide the building blocks (APIs, SDKs, and UI widgets) to develop end-user applications quickly. By handling the "under-the-hood" infrastructure—like scalability and security—they allow developers to focus on the specific **user experience** and business logic rather than the plumbing of the network.

5. Security and Encryption

Security is the most critical role. Platforms provide **end-to-end encryption**

Q 8. Sensors must report moisture data to a central cloud server Analyze the protocol requirements for this application by addressing the following:

1. Selection Factor Analysis
2. Architectural Differentiation
3. Communication Overhead
4. Ans: **Selection Factor Analysis:** Deconstruct how the specific constraints of **Power Consumption**, **Network Reliability**, and **Data Payload Size** in this forest scenario dictate the choice of an IoT protocol. Explain the relationship between these factors and the device's operational lifespan.

5. **Architectural Differentiation :** Differentiate between **CoAP** and **MQTT** for this specific deployment. Analyze how CoAP's **UDP-based Request/Response** model compares to MQTT's **TCP-based Publish/Subscribe** model in terms of handling "intermittent connectivity" and "battery conservation."

6. **Communication Overhead :** Explain why using a **binary-based protocol** (like CoAP or MQTT) is a more analytical choice for this high-density deployment than a text-based protocol like HTTP.

Key Analytical Points:

- **Factors:** Must link **Power** to the protocol's "overhead" (header size), **Reliability** to the transport layer (TCP vs UDP), and **Latency** to the communication pattern (Push vs Pull).

- **CoAP vs MQTT:**
  - **CoAP:** Better for "sleeping" devices (UDP is connectionless) and low-power headers, but harder to manage behind NAT/Firewalls.
  - **MQTT:** Better for reliable delivery (TCP/QoS levels) and many-to-many communication but keeps the connection "alive" (Heartbeat/Keep-alive), which can drain battery faster.
- **Binary vs Text:** Analyzing that text (HTTP/JSON) is "human-readable" but "machine-expensive," whereas binary is "bandwidth-efficient" for 2,000 nodes.

Q 9 Serial-based SCADA system to a modern Ethernet-based infrastructure. transport serial DNP3 over IP.

Ans: The IEEE 1815-2012 specification describes how the DNP3 protocol implementation must be adapted to run either over TCP (recommended) or UDP. This specification defines connection management between the DNP3 protocol and the I Players, as shown in Figure Connection management links the DNP3 layers with the IP layers in addition to the configuration parameters and methods necessary for implementing the network connection. The IP layers appear transparent to the DNP3 layers a piece of the protocol stack in one station logically communicates with the respective part in the other. This means that the DNP3 endpoints or devices are not aware of the underlying IP transport that is occurring.

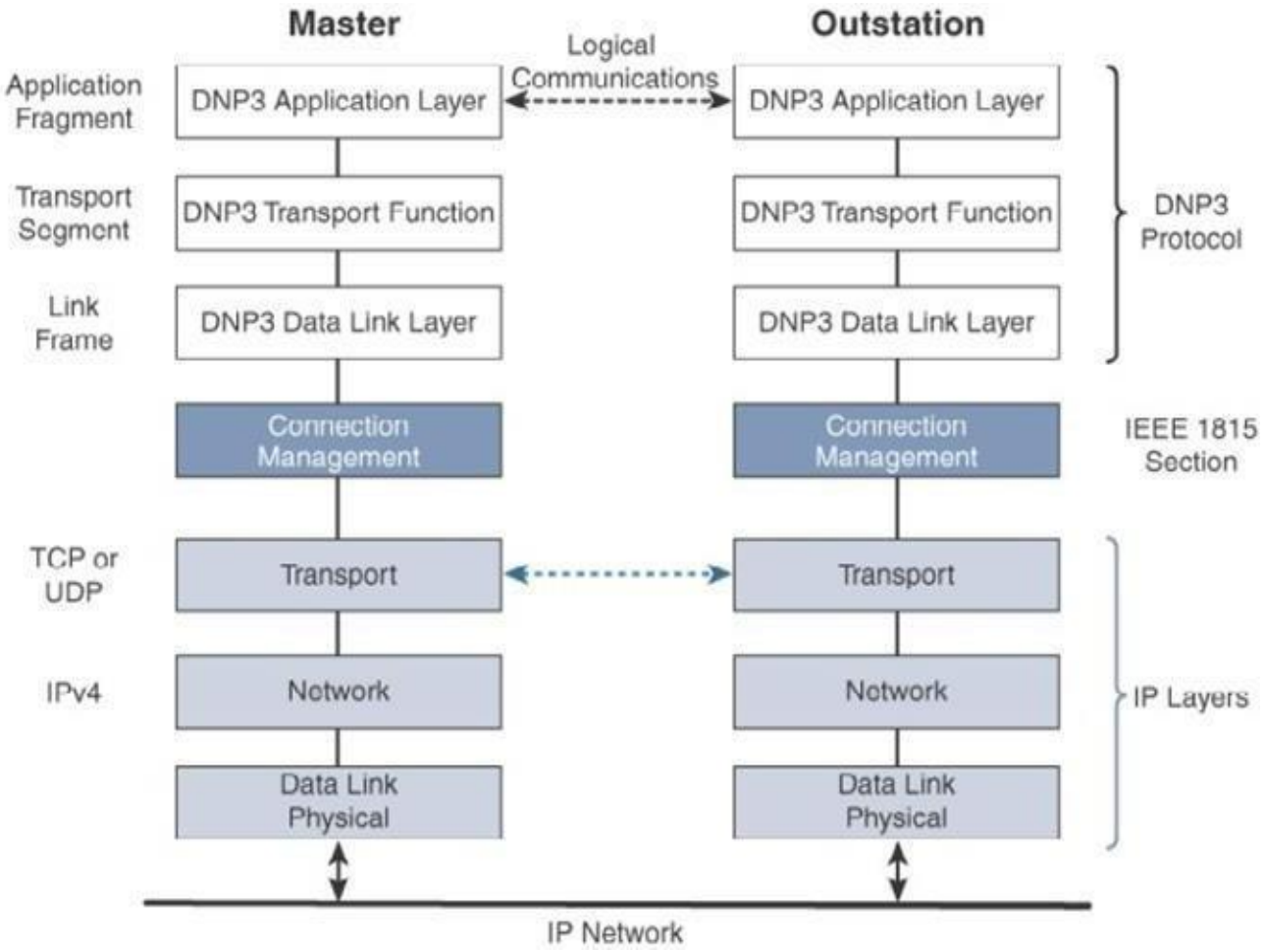


Figure 1 :Protocol Stack for Transporting Serial DNP3 SCADA over IP

In Figure 1, the master side initiates connections by performing a TCP active open. The outstation listens for a connection request by performing a TCP passive open. *Dual endpoint* is defined as a process that can both listen for connection requests and perform an active open on the channel if required. Master stations may parse multipleDNP3 data link layer frames from a single UDP datagram, while DNP3 data link layer frames can not span multiple UDP data grams. Single or multiple connections to the master may get established while a TCP keep alive timer monitors the status of the connection. Keep alive messages are implemented as DNP3 data link layer status requests. If a response is not received to a keep alive message, the connection is deemed broken, and the appropriate action is taken.

Q 10. Write short note on SCADA?

Ans: Core Components

A typical SCADA system consists of four primary elements:

- **Field Devices:** Sensors and actuators that interact directly with machines to collect data (temperature, pressure) or perform physical actions (opening a valve).
- **Controllers (RTUs and PLCs):**
  - **Remote Terminal Units (RTUs):** Small computers that gather data from field devices and transmit it to the central station, often used in remote locations.
  - **Programmable Logic Controllers (PLCs):** Ruggedized computers used for local machine control and logic execution.
- **Communication Infrastructure:** The network (wired like Ethernet or wireless like radio/cellular) that connects field controllers to the central server.
- **Human-Machine Interface (HMI):** A dashboard or graphical user interface (GUI) that presents processed data to operators, allowing them to monitor status, receive alarms, and issue manual commands.

Key Functions

- **Data Acquisition:** Continuously gathering real-time information from sensors.
- **Supervisory Control:** Enabling operators to adjust set-points or override automated actions remotely.
- **Alarm Management:** Notifying operators immediately when system parameters exceed safe limits.

- **Data Logging & Historian:** Recording historical data for future trend analysis, reporting, and predictive maintenance.

Q 11. Explain an IoT software Platform-REST.

ANS: A **REST (Representational State Transfer)** platform refers to an architectural style that uses standard web technologies—primarily **HTTP**—to enable communication between devices, cloud servers, and applications.

The platform follows a **Request-Response** model where a "Client" (e.g., a smart sensor or mobile app) initiates a request, and a "Server" (the IoT platform) provides a response.

- **Standard Methods (Verbs):** Interactions use standard HTTP commands to manage device data:
    - **GET**: Retrieve data (e.g., fetch current temperature).
    - **POST**: Create a new record or send a command (e.g., log a new sensor reading).
    - **PUT / PATCH**: Update an existing state (e.g., change a thermostat setting).
    - **DELETE**: Remove a resource (e.g., unregister a device).
- **Data Format:** Information is typically exchanged in **JSON**, a lightweight, text-based format that is easy for both humans to read and machines to parse.

Key Characteristics

- **Statelessness:** The server does not store information about the client's session. Every request must contain all the information needed to process it (e.g., authentication tokens), which makes the platform highly **scalable**.
- **Interoperability:** Because it uses standard web protocols, RESTful platforms work across any hardware, operating system, or programming language.
- **Familiarity:** It uses the same technology as the World Wide Web, making it easy for web developers to build and debug IoT applications

---

✅ MD5 (Message Digest Algorithm 5)

Definition:

MD5 (Message Digest 5) is a **cryptographic hash function** that converts any input message into a **fixed-length 128-bit hash value**, mainly used to ensure **data integrity**.

It was developed by Ron Rivest in 1991.

Basic Concept:

- Input: Any size data (text, file, etc.)
- Output: Fixed **128-bit hash (message digest)**
- Even a small change in input → completely different output

Working of MD5 (Step-by-Step):

1. Padding

- Message is extended so its length becomes **448 mod 512**
- Extra bits are added (1 followed by 0s)

---

2. Append Length

- Original message length (in bits) is added at the end
- Final size becomes multiple of **512 bits**

---

3. Initialize Buffers

- Four 32-bit variables are initialized:
  - A
  - B
  - C
  - D

---

4. Process Message in Blocks

- Message is divided into **512-bit blocks**
- Each block is processed in **4 rounds (16 operations each = 64 operations)**

- Uses logical functions like:
    - AND, OR, XOR, NOT

---

5. Output Generation

- Final values of A, B, C, D are combined
- Produces a **128-bit hash value**

Diagram (Important for Exam):

Input Message → Padding → Append Length → Initialize Buffers → Process Blocks → 128-bit Hash Output

---

Example:

- Input: hello
- Output: 5d41402abc4b2a76b9719d911017c592

👉 If input changes to Hello → completely different hash

Properties of MD5:

- Fixed output length (128-bit)
- Deterministic (same input → same output)
- One-way function
- Fast computation

Advantages:

- Simple and fast
- Easy to implement
- Used for **file integrity checking**

Disadvantages:

- Vulnerable to **collision attacks**
- Not secure for modern cryptography
- Can be broken by attackers

Applications:

- File verification (downloads)
- Checksums
- Digital fingerprinting (older systems)

Conclusion:

MD5 is a fast hashing algorithm useful for **data integrity**, but due to security weaknesses, it is **no longer recommended for cryptographic security purposes**.

✅ MAC (Message Authentication Code)

Definition:

MAC (Message Authentication Code) is a **cryptographic technique** used to ensure **data integrity and authentication** by generating a fixed-size code using a **secret key and message**.

Basic Idea (Simple Understanding 😉):

👉 Sender and receiver share a **secret key**
👉 Sender creates a **MAC value (tag)** from message
👉 Receiver checks MAC to verify:

- Message is not changed
- Message is from valid sender

Working of MAC (Step-by-Step):

1. At Sender Side

1. Take message (M)
2. Apply MAC algorithm with **secret key (K)**
3. Generate MAC value (Tag)
4. Send:
    - Message + MAC

---

2. At Receiver Side

1. Receive message + MAC

2. Apply same MAC function using same key
3. Generate new MAC
4. Compare both MAC values
5. If same → **Valid message**, else **Rejected**

Diagram (Important for Exam):

Sender:

Message + Secret Key → MAC Algorithm → MAC → Send (Message + MAC)

Receiver:

Message + Secret Key → MAC Algorithm → MAC → Compare → Accept / Reject

---

Formula:

MAC = F(K, M)

Where:

- K = Secret Key
- M = Message

Properties:

- Provides **Integrity** (data not modified)
- Provides **Authentication** (sender verified)
- Uses **secret key**
- Fixed-size output

Advantages:

- Simple and efficient
- Protects against data tampering
- Faster than digital signatures

Disadvantages:

- Does not provide **non-repudiation**

- Requires secure key sharing
- If key is leaked → system compromised

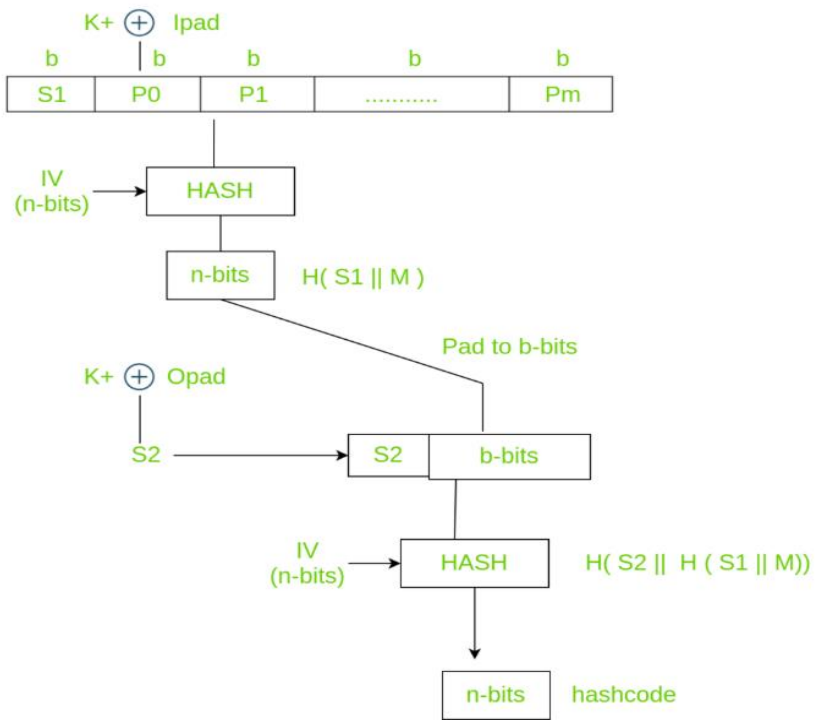
Applications:

- Secure communication
- Banking systems
- Network protocols
- Data transmission

Conclusion:

MAC is an important security mechanism used to ensure **data integrity and authentication**, but it requires **secure key management** and does not provide non-repudiation.

✅ HMAC (Hash-based Message Authentication Code)



Definition:

HMAC (Hash-based Message Authentication Code) is a cryptographic technique that uses a **secret key and a hash function** to provide **data integrity and authentication**.

😊 Basic Idea (Simple Understanding):

👉 Normal MAC = message + key
👉 HMAC = **hash function + secret key + message**

It improves security by using **two layers of hashing**.

HMAC Formula (Important):

🔷 HMAC Formula

$$HMAC(K, M) = H\big((K \oplus opad) \,||\, H((K \oplus ipad) \,||\, M)\big)$$

Where:

- K = Secret key
- M = Message
- H = Hash function (like MD5/SHA)
- opad = Outer padding
- ipad = Inner padding
- ⊕ = XOR operation
- || = Concatenation

🔄 Full Process of HMAC (Step-by-Step)

1. Key Preparation

- If key is long → shorten it
- If short → pad with zeros

---

2. Create Inner Key

- K ⊕ ipad (inner padding)

---

3. Inner Hashing

- Concatenate with message:
  👉 (K ⊕ ipad) || Message
- Apply hash function → **Inner Hash**

---

4. Create Outer Key

- K ⊕ opad (outer padding)

---

5. Outer Hashing

- Concatenate:
  ☞ (K ⊕ opad) || Inner Hash
- Apply hash function again

---

6. Final Output

- Result = **HMAC value (MAC tag)**

Diagram (Very Important for Exam):

Step 1: K ⊕ ipad → + Message → Hash → Inner Hash

Step 2: K ⊕ opad → + Inner Hash → Hash → Final HMAC

Properties:

- Provides **Integrity**
- Provides **Authentication**
- Resistant to attacks
- Uses secret key

Advantages:

- More secure than simple MAC
- Protects against collision attacks
- Widely used in real systems

Disadvantages:

- Requires secure key sharing
- Slightly slower than simple MAC

Applications:

- HTTPS (SSL/TLS)

- API authentication
- Secure communication

Conclusion:

HMAC is a secure and efficient method for ensuring **message integrity and authentication**, widely used in modern security protocols.

✅ CMAC (Cipher-based Message Authentication Code)

Definition:

CMAC (Cipher-based Message Authentication Code) is a cryptographic technique that uses a **block cipher (like AES)** along with a **secret key** to provide **data integrity and authentication**.

😊 Basic Idea (Simple Understanding):

👉 Instead of using a hash function (like HMAC), CMAC uses **encryption algorithm (AES)**
👉 It generates a **MAC value (tag)** to verify the message

🔄 Working of CMAC (Step-by-Step)

1. Key Generation

- A secret key **K** is shared between sender and receiver

---

2. Subkey Generation

- Using AES, two subkeys are generated:
  - **K1**
  - **K2**

---

3. Message Preparation

- Message is divided into **fixed-size blocks (128-bit)**
- If last block is incomplete → padded

---

4. Processing Blocks

- Each block is processed using **XOR and AES encryption**

- Intermediate result is updated after each block

---

5. Final Block Processing

- If last block is complete → XOR with **K1**
- If incomplete → XOR with **K2**

---

6. Final Output

- Final encrypted result = **CMAC (MAC tag)**

📊 **Diagram (Important for Exam):**

Message → Divide into Blocks → XOR → AES Encrypt → Repeat for all blocks

↓

Final Block + K1/K2

↓

AES Encrypt

↓

CMAC Output

🔑 **Properties:**

- Provides **Integrity**
- Provides **Authentication**
- Uses **block cipher (AES)**
- Fixed-length output

✅ **Advantages:**

- More secure than simple MAC
- Based on strong encryption (AES)
- Suitable for hardware implementation
- Standardized and widely accepted

❌ **Disadvantages:**

- Slower than simple MAC
- Requires key management
- Depends on cipher strength

📌 **Applications:**

- Secure communication systems
- Banking and financial systems
- Wireless security (e.g., LTE)
- Data authentication

□ Conclusion:

CMAC is a secure and efficient authentication method that uses **block cipher encryption** to ensure **data integrity and authentication**, making it suitable for modern secure systems.

✅ MD5 (Message Digest Algorithm 5)

Definition:

MD5 (Message Digest 5) is a **cryptographic hash function** that converts any input message into a **fixed-length 128-bit hash value**, mainly used to ensure **data integrity**.

It was developed by Ron Rivest in 1991.

Basic Concept:

- Input: Any size data (text, file, etc.)
- Output: Fixed **128-bit hash (message digest)**
- Even a small change in input → completely different output

Working of MD5 (Step-by-Step):

1. Padding

- Message is extended so its length becomes **448 mod 512**
- Extra bits are added (1 followed by 0s)

---

2. Append Length

- Original message length (in bits) is added at the end
- Final size becomes multiple of **512 bits**

---

3. Initialize Buffers

- Four 32-bit variables are initialized:
  - A
  - B
  - C
  - D

---

4. Process Message in Blocks

- Message is divided into **512-bit blocks**
- Each block is processed in **4 rounds (16 operations each = 64 operations)**

- Uses logical functions like:
    - AND, OR, XOR, NOT

---

5. Output Generation

- Final values of A, B, C, D are combined
- Produces a **128-bit hash value**

Diagram (Important for Exam):

Input Message → Padding → Append Length → Initialize Buffers → Process Blocks → 128-bit Hash Output

---

Example:

- Input: hello
- Output: 5d41402abc4b2a76b9719d911017c592

👉 If input changes to Hello → completely different hash

Properties of MD5:

- Fixed output length (128-bit)
- Deterministic (same input → same output)
- One-way function
- Fast computation

Advantages:

- Simple and fast
- Easy to implement
- Used for **file integrity checking**

Disadvantages:

- Vulnerable to **collision attacks**
- Not secure for modern cryptography
- Can be broken by attackers

Applications:

- File verification (downloads)
- Checksums
- Digital fingerprinting (older systems)

Conclusion:

MD5 is a fast hashing algorithm useful for **data integrity**, but due to security weaknesses, it is **no longer recommended for cryptographic security purposes**.

✅ MAC (Message Authentication Code)

Definition:

MAC (Message Authentication Code) is a **cryptographic technique** used to ensure **data integrity and authentication** by generating a fixed-size code using a **secret key and message**.

Basic Idea (Simple Understanding 😉):

👉 Sender and receiver share a **secret key**
👉 Sender creates a **MAC value (tag)** from message
👉 Receiver checks MAC to verify:

- Message is not changed
- Message is from valid sender

Working of MAC (Step-by-Step):

1. At Sender Side

1. Take message (M)
2. Apply MAC algorithm with **secret key (K)**
3. Generate MAC value (Tag)
4. Send:
    - Message + MAC

---

2. At Receiver Side

1. Receive message + MAC

2. Apply same MAC function using same key
3. Generate new MAC
4. Compare both MAC values
5. If same → **Valid message**, else **Rejected**

Diagram (Important for Exam):

Sender:

Message + Secret Key → MAC Algorithm → MAC → Send (Message + MAC)

Receiver:

Message + Secret Key → MAC Algorithm → MAC → Compare → Accept / Reject

---

Formula:

MAC = F(K, M)

Where:

- K = Secret Key
- M = Message

Properties:

- Provides **Integrity** (data not modified)
- Provides **Authentication** (sender verified)
- Uses **secret key**
- Fixed-size output

Advantages:

- Simple and efficient
- Protects against data tampering
- Faster than digital signatures

Disadvantages:

- Does not provide **non-repudiation**

- Requires secure key sharing
- If key is leaked → system compromised

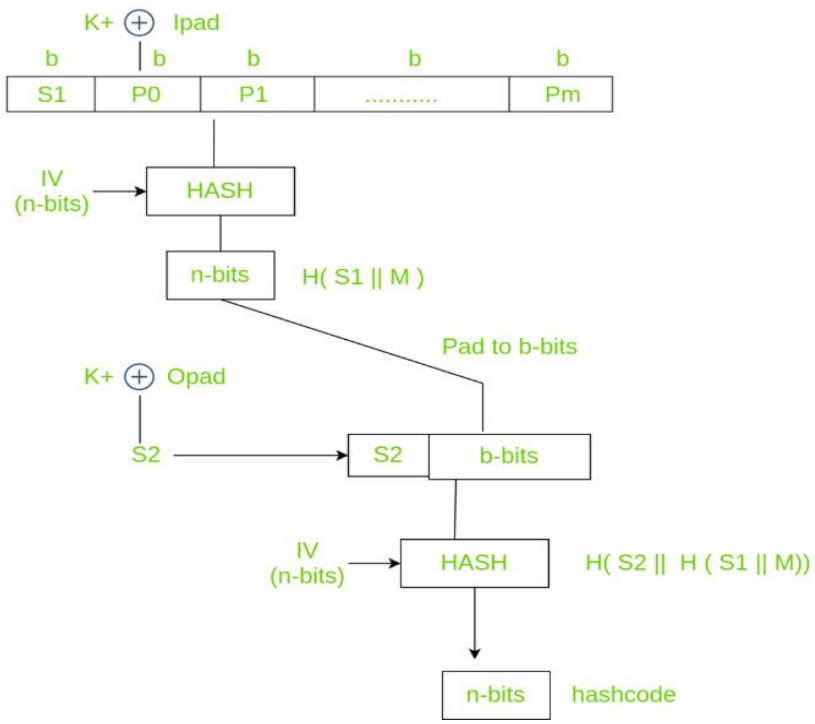
Applications:

- Secure communication
- Banking systems
- Network protocols
- Data transmission

Conclusion:

MAC is an important security mechanism used to ensure **data integrity and authentication**, but it requires **secure key management** and does not provide non-repudiation.

✅ HMAC (Hash-based Message Authentication Code)



Definition:

HMAC (Hash-based Message Authentication Code) is a cryptographic technique that uses a **secret key and a hash function** to provide **data integrity and authentication**.

😊 Basic Idea (Simple Understanding):

👉 Normal MAC = message + key
👉 HMAC = **hash function + secret key + message**

It improves security by using **two layers of hashing**.

HMAC Formula (Important):

🔷 HMAC Formula

$$HMAC(K, M) = H\big((K \oplus opad) \,||\, H((K \oplus ipad) \,||\, M)\big)$$

Where:

- K = Secret key
- M = Message
- H = Hash function (like MD5/SHA)
- opad = Outer padding
- ipad = Inner padding
- ⊕ = XOR operation
- || = Concatenation

🔄 Full Process of HMAC (Step-by-Step)

1. Key Preparation

- If key is long → shorten it
- If short → pad with zeros

---

2. Create Inner Key

- K ⊕ ipad (inner padding)

---

3. Inner Hashing

- Concatenate with message:
  👉 (K ⊕ ipad) || Message
- Apply hash function → **Inner Hash**

---

4. Create Outer Key

- K ⊕ opad (outer padding)

---

5. Outer Hashing

- Concatenate:
  ☞ (K ⊕ opad) || Inner Hash
- Apply hash function again

---

6. Final Output

- Result = **HMAC value (MAC tag)**

Diagram (Very Important for Exam):

Step 1: K ⊕ ipad → + Message → Hash → Inner Hash

Step 2: K ⊕ opad → + Inner Hash → Hash → Final HMAC

Properties:

- Provides **Integrity**
- Provides **Authentication**
- Resistant to attacks
- Uses secret key

Advantages:

- More secure than simple MAC
- Protects against collision attacks
- Widely used in real systems

Disadvantages:

- Requires secure key sharing
- Slightly slower than simple MAC

Applications:

- HTTPS (SSL/TLS)

- API authentication
- Secure communication

Conclusion:

HMAC is a secure and efficient method for ensuring **message integrity and authentication**, widely used in modern security protocols.

✅ CMAC (Cipher-based Message Authentication Code)

Definition:

CMAC (Cipher-based Message Authentication Code) is a cryptographic technique that uses a **block cipher (like AES)** along with a **secret key** to provide **data integrity and authentication**.

😊 Basic Idea (Simple Understanding):

👉 Instead of using a hash function (like HMAC), CMAC uses **encryption algorithm (AES)**
👉 It generates a **MAC value (tag)** to verify the message

🔄 Working of CMAC (Step-by-Step)

1. Key Generation

- A secret key **K** is shared between sender and receiver

---

2. Subkey Generation

- Using AES, two subkeys are generated:
  - **K1**
  - **K2**

---

3. Message Preparation

- Message is divided into **fixed-size blocks (128-bit)**
- If last block is incomplete → padded

---

4. Processing Blocks

- Each block is processed using **XOR and AES encryption**

- Intermediate result is updated after each block

---

5. Final Block Processing

- If last block is complete → XOR with **K1**
- If incomplete → XOR with **K2**

---

6. Final Output

- Final encrypted result = **CMAC (MAC tag)**

📊 **Diagram (Important for Exam):**

Message → Divide into Blocks → XOR → AES Encrypt → Repeat for all blocks

↓

Final Block + K1/K2

↓

AES Encrypt

↓

CMAC Output

🔑 **Properties:**

- Provides **Integrity**
- Provides **Authentication**
- Uses **block cipher (AES)**
- Fixed-length output

✅ **Advantages:**

- More secure than simple MAC
- Based on strong encryption (AES)
- Suitable for hardware implementation
- Standardized and widely accepted

❌ **Disadvantages:**

- Slower than simple MAC
- Requires key management
- Depends on cipher strength

📌 **Applications:**

- Secure communication systems
- Banking and financial systems
- Wireless security (e.g., LTE)
- Data authentication

□ Conclusion:

CMAC is a secure and efficient authentication method that uses **block cipher encryption** to ensure **data integrity and authentication**, making it suitable for modern secure systems.

✅ 8. TCP/IP Layer Vulnerabilities (5 Marks)

Definition:

TCP/IP model has multiple layers, and each layer has **security weaknesses** that attackers can exploit.

Layer-wise Vulnerabilities:

Layer	Vulnerabilities
Application Layer	Malware, phishing, data theft
Transport Layer	SYN flood, port scanning
Network Layer	IP spoofing
Data Link Layer	ARP spoofing

Explanation:

- Each layer performs different functions, so attacks target specific layers
- Lack of built-in security in TCP/IP makes it vulnerable

Conclusion:

TCP/IP vulnerabilities can lead to serious attacks; hence additional security mechanisms are required.

✅ 9. IP Spoofing (5 Marks)

Definition:

IP Spoofing is a technique where an attacker **creates fake IP addresses** to hide identity or impersonate another system.

Working:

1. Attacker modifies source IP address
2. Sends packet to target
3. Target believes it is from a trusted source

Uses / Attacks:

- DDoS attacks
- Bypassing security
- Session hijacking

Prevention:

- Packet filtering
- Firewalls
- Authentication methods

Conclusion:

IP spoofing is dangerous as it hides attacker identity and enables network attacks.

✅ 10. ARP Spoofing (5 Marks)

Definition:

ARP Spoofing is an attack where an attacker sends **fake ARP messages** to link their MAC address with a legitimate IP address.

Working:

1. Attacker sends fake ARP reply
2. Victim updates ARP table
3. Traffic gets redirected to attacker

Effects:

- Man-in-the-middle attack
- Data theft
- Network disruption

Prevention:

- Static ARP entries
- ARP monitoring tools
- Secure protocols

Conclusion:

ARP spoofing compromises network communication by redirecting traffic.

✅ 11. IPSec (Internet Protocol Security) (5 Marks)

Definition:

IPSec is a protocol suite used to secure **IP communication** by providing **encryption and authentication**.

Modes:

1. **Transport Mode** → Encrypts only data
2. **Tunnel Mode** → Encrypts entire packet

Protocols:

- AH (Authentication Header)
- ESP (Encapsulating Security Payload)

Features:

- Confidentiality
- Integrity
- Authentication

Conclusion:

IPSec ensures secure communication over networks, especially in VPNs.

✅ 12. PGP (Pretty Good Privacy) (5 Marks)

Definition:

PGP is a security technique used to **encrypt emails and files** using both **symmetric and asymmetric encryption**.

Developed by Phil Zimmermann.

Working:

1. Message is hashed
2. Encrypted using symmetric key
3. Key encrypted using receiver's public key
4. Receiver decrypts and verifies

Features:

- Confidentiality
- Integrity
- Authentication
- Non-repudiation

Applications:

- Email security
- File encryption

Conclusion:

PGP provides strong security by combining encryption and digital signatures.

✅ 1. Password-Based Authentication

Definition:

Password-Based Authentication is a method in which a user verifies identity by entering a **username and password**, which is checked by the system.

Working / Steps:

1. User enters **username and password**
2. System retrieves stored **hashed password** from database
3. Entered password is **converted into hash** using same algorithm
4. System compares entered hash with stored hash
5. If both match → **Access Granted**, else **Access Denied**

Diagram:

User → Enter Password → Hash Function → Compare with Stored Hash → Access

---

Advantages:

- Simple and easy to implement
- Low cost and widely used
- No special hardware required

Disadvantages:

- Weak passwords can be guessed easily
- Vulnerable to **brute-force and dictionary attacks**
- Password theft (phishing, keylogging)
- Users often reuse passwords

Applications:

- Email accounts
- ATM PIN systems
- Social media login

Conclusion:

Password-based authentication is widely used but less secure compared to modern methods; hence it is often combined with **multi-factor authentication**.

✅ 2. Challenge-Response Authentication

Definition:

Challenge-Response Authentication is a secure method where the server sends a **random challenge**, and the client responds using a **secret key**, proving identity without sending the password.

Working / Steps:

1. User sends login request
2. Server generates and sends a **random challenge (nonce)**
3. Client encrypts challenge using **secret key/password**
4. Client sends encrypted response back to server
5. Server performs same operation and verifies response
6. If matched → **Access Granted**

Diagram:

Server → Challenge (Random Number)

Client → Encrypt using Secret Key → Response

Server → Verify → Access

Advantages:

- Prevents **replay attacks**
- Password is not transmitted directly
- More secure than password-based authentication

Disadvantages:

- More complex to implement
- Requires secure key storage
- Needs synchronization between client and server

Applications:

- One-Time Password (OTP) systems
- Online banking authentication
- Secure network protocols

Conclusion:

Challenge-response authentication provides better security than password-based systems by avoiding direct transmission of sensitive data.

✅ 3. Digital Signature using RSA

Definition:

A digital signature is a cryptographic technique used to verify the **authenticity, integrity, and non-repudiation** of a message using RSA algorithm.

It is based on the work of:

- Ron Rivest
- Adi Shamir
- Leonard Adleman

Process of Digital Signature

◈ 1. Signing Process (Sender Side)

1. Sender creates message
2. Apply **hash function** → message digest
3. Encrypt digest using **private key**
4. This becomes the **digital signature**
5. Send:
    - Message + Digital Signature

◈ 2. Verification Process (Receiver Side)

1. Receiver gets message + signature
2. Decrypt signature using **public key**

3. Compute hash of received message
4. Compare both hashes
5. If equal → **Valid Signature**

Diagram (important for exam):

Sender:

Message → Hash → Encrypt (Private Key) → Signature → Send

Receiver:

Message → Hash

Signature → Decrypt (Public Key)

Compare → Verified / Not Verified

Properties Provided:

- **Authentication** → verifies sender
- **Integrity** → message not altered
- **Non-repudiation** → sender cannot deny

Advantages:

- Highly secure
- Widely used in e-commerce
- Ensures trust

Disadvantages:

- Computationally expensive
- Requires key management
- Private key must be protected

Applications:

- Digital certificates
- Online banking

- Email security
- E-commerce transactions